RetailOps order model

A study guide to how the store-only order model decides what each shop orders every morning: the theory, all of the maths, and one real product followed for eight weeks.

Who it is for: anyone on the team, or anyone at the viva, who has done school algebra and a little probability. Every idea is explained from scratch before it is used.

What it covers: the main order model **store_ilp** (2-day forecaster → newsvendor targets → one-day store ILP with a 90% service floor and a spoilage cap → shadow prices and what-ifs), the simple rules it is compared against, and its limits.

Data used: the 8-week test window only (1 Nov to 26 Dec 2015) and the forecasts that already exist for it. Nothing from the sealed holdout (27 Dec 2015 onward) is used anywhere in this guide.

Main result (8 test weeks, 10 stores, 120 products)	Value
Fill rate (share of customer demand served)	**94.96%**
Units thrown away (waste)	**7,015**
Total cost: money spent + penalty for lost sales	**932,094**
Best simple rule (top-up to forecast): fill / waste / cost	97.9% / 73,861 / 1,012,368

CS2105 Optimization Techniques · IIT Patna · RetailOps team · 2 November 2026

Money is in the dataset's price units (the M5 data's prices are US dollars).

Contents

Contents 2

What the order model does 4

The shop, the timing and the scorecard 5

- The setting . . . 5
- The timing of one day . . . 5
- The scorecard . . . 5

Why the forecast is four numbers, not one 6

- Quantiles in one sentence . . . 6
- How a quantile line is learned: the pinball loss . . . 6
- Our forecaster . . . 6

How much stock is the right amount? 7

- The two costs . . . 7
- The marginal argument (the whole proof) . . . 7
- What the ratio is for our products . . . 7
- Finding the 0.627 line between q50 and q80 . . . 8

Order-up-to, the spoilage cap and how waste is counted 9

- The order-up-to rule . . . 9
- The spoilage cap (perishables) . . . 9
- How the replay counts waste . . . 9
- The service floor . . . 9

From 'what is best' to a model a computer can solve 10

- A tiny LP you can draw . . . 10
- Why whole numbers make it harder . . . 10
- Two modelling tricks the store ILP uses . . . 11

The one-day store ILP, line by line 12

- Decision variables . . . 12
- Objective: minimise today's total cost . . . 12
- Constraints . . . 12
- Size and speed . . . 13

What is one more unit of shelf worth? 14

- Complementary slackness . . . 14
- How we get shadow prices from an ILP . . . 14
- Results at today's settings . . . 14
- What-ifs: tightening one limit at a time . . . 14

Worked example: FOODS_3_252 at store CA_1 16

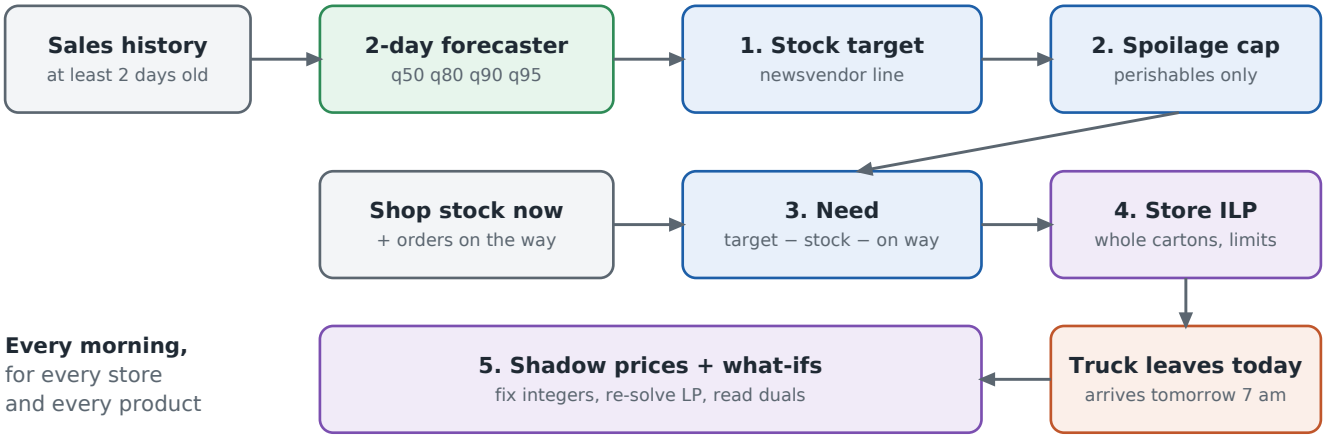
The product . . . 16
One morning in full: Friday 6 November 2015 . . . 16
Four more days worth looking at . . . 17
The whole eight weeks . . . 18
Same product, same 8 weeks, every method . . . 18
How it compares, and why one ILP beats patching **19**
All methods, 8 test weeks . . . 19
Idea P1: patching rules one by one vs one ILP . . . 19
What the model does not capture **20**
Questions you may be asked **21**
Glossary . . . 21

PART 1 · THE TWO-MINUTE VERSION

What the order model does

Imagine you run 10 grocery shops. Every morning you must decide, for each of 120 products in each shop, how many cartons to order from the warehouse. What you order today arrives tomorrow morning. Order too little and customers leave empty-handed (a **lost sale**). Order too much of a fresh product and it goes bad (**waste**). Order too much of a long-life product and it just sits on the shelf, tying up money.

The order model makes that decision for all 1,200 store-products at once, every morning, in five steps:



Step	In plain words	Maths used
Forecast	Guess today's and tomorrow's sales, not as one number but as four levels: a middle guess (q50) and three 'safe' guesses (q80, q90, q95).	Quantile regression (Part 3)
1. Stock target	Decide how many units to have for today + tomorrow by weighing the cost of running out against the cost of having too many.	Newsvendor model (Part 4)
2. Spoilage cap	Never hold more of a fresh product than you expect to sell before it goes bad.	Simple bound (Part 5)
3. Need	Take away what the shop already has and what is already coming.	Order-up-to rule (Part 5)
4. Store ILP	Turn the needs into whole cartons and trucks, while respecting truck, shelf and budget limits, a 90% service floor and the spoilage cap, at the lowest total cost.	Integer linear programming (Parts 6–7)
5. Shadow prices	Ask 'how much would one more unit of shelf / truck / budget be worth?'	LP duality (Part 8)

The headline

Over 8 test weeks this serves 94.96% of demand, throws away 7,015 units and costs 932,094 in total. The best simple rule serves more (97.9%) but throws away ten times as much (73,861 units) and costs 8% more (1,012,368). The model deliberately accepts a few more lost sales to avoid a lot of waste, because the numbers say that is cheaper.

The shop, the timing and the scorecard

The setting

- **Stores and products.** 10 Walmart stores in California, Texas and Wisconsin from the M5 dataset, 120 food products each. 20 products are **perishable** (shelf life 2 to 5 days); 100 are **long-life**.
- **Store-only design.** We assume the warehouse always has enough stock and delivers every store order the next morning. The warehouse part of the model still exists behind a switch (*replay.warehouse: off*) for later.
- **Cartons.** Products come in cartons (cases) of 6, 12, 24 or 48 units depending on how fast they sell. A shop can only order whole cartons, and at least one carton when it orders at all.
- **Trucks.** A truck carries up to 400 cartons and each trip costs 40. Each order line (one product on one order) costs 0.50 of paperwork.
- **Calendar.** Stores are closed on 25 December, so nothing is sold or delivered that day. An order on 24 Dec would arrive on the 25th, so no order is placed on 24 Dec, and the 23 Dec order must cover the 23rd to the 25th.

The timing of one day

Time	What happens
Early morning	Yesterday's order arrives and goes on the shelf. The shop now has its **morning stock**.
Morning	The model sees the stock, the orders on the way and the forecast for today and tomorrow, and places today's order.
During the day	Customers buy. Sold = the smaller of demand and stock. Anything wanted but missing is a **lost sale**.
Evening	Fresh stock that will not sell before it goes bad is thrown away (**waste**). What is left is the next morning's opening stock.
Next morning	Today's order arrives. Repeat.

The cover window: an order placed today arrives tomorrow, and the next order arrives the day after. So today's order (together with what is already in the shop) has to last **today and tomorrow**. That is why the forecaster only needs to look 2 days ahead.

The scorecard

Measure	Meaning	Formula
Fill rate	Share of customer demand actually served	units sold ÷ units demanded
Lost units	Demand that found an empty shelf	Σ max(demand − stock, 0)
Waste	Fresh units thrown away	see Part 5
Net cost	Money spent: buying goods, trucks, order lines, holding, disposal	sum of cash items
Lost-sale penalty	A price on each lost unit: 1.5 × its selling price (lost profit + an unhappy customer)	1.5 × price × lost units
Total cost	The number every method is ranked by	net cost + lost-sale penalty

> The 1.5 × price penalty is an assumption, not a fact from the data. Part 8 shows what happens when it is 1.0 or 2.0.

Why the forecast is four numbers, not one

Nobody knows exactly how many loaves will sell tomorrow. If a product sells about 35 a day, some days it sells 20 and some days 55. A single guess like '35' hides this spread, and the spread is exactly what decides how much safety stock to hold.

Quantiles in one sentence

The **q-quantile** of tomorrow's sales is the number that sales stay at or below on a share q of days. So **q50** is the middle guess (half the days below, half above), **q80** is a level sales stay under on 80 days out of 100, and **q95** on 95 days out of 100.

$$P(\text{ demand} \le q_{\tau}) = \tau \text{ for } \tau = 0.50, 0.80, 0.90, 0.95$$

How a quantile line is learned: the pinball loss

An ordinary forecast is trained to make the average error small. A quantile line is trained with a lopsided error called the **pinball loss**. For the q80 line, being 1 unit too low costs 0.8, being 1 unit too high costs only 0.2. The line that makes the total smallest ends up above sales on about 80% of days.

$$L_{\tau}(y, q) = \tau \cdot (y - q) \text{ if } y \ge q, (1 - \tau) \cdot (q - y) \text{ if } y < q$$

Why does that work? Move the line up by a tiny step. On days where sales were above the line (share $1 - F$), the loss falls by τ per unit; on days below (share F) it rises by $(1 - \tau)$. The total stops improving when $\tau(1 - F) = (1 - \tau)F$, which gives $F = \tau$. Keep this argument in mind: Part 4 uses exactly the same trick.

Our forecaster

- One gradient-boosted tree model per line (q50, q80, q90, q95), trained on 2013 to Oct 2015.
- Inputs: sales 2, 7, 14, 21 and 28 days earlier, 7- and 28-day averages, day of week, events, SNAP days, prices, store and product.
- **Rule R2: every store input is at least 2 days old.** The forecast for today and tomorrow is made only from sales up to the day before yesterday, so it can never peek at the answer.
- Accuracy: the middle line is off by 3.82 units per store-product-day on average (validation fold 2), 8–10% better than before. The upper lines cover about 81%, 90% and 94.6% of days, close to the 80/90/95% they promise.

> The order model only reads **q50, q80 and q95**. q90 is produced but unused by the store model.

How much stock is the right amount?

The newsvendor problem is the oldest model in inventory theory. A boy sells newspapers. In the morning he buys Q papers; in the evening unsold papers are worthless. Demand D is uncertain. How many should he buy?

The two costs

- **C_u (cost of under-stocking)**: what one missing unit costs. For us, the lost-sale penalty: C_u = 1.5 × price.
- **C_o (cost of over-stocking)**: what one unit too many costs. For a perishable it may be thrown away, so we lose its buy cost plus 5% to dispose of it: C_o = buy cost × 1.05. Buy cost is 85% of the selling price.

The marginal argument (the whole proof)

Suppose you already stock Q units and think about adding **one more**.

- That extra unit gets sold only if demand is bigger than Q. Chance: $P(D > Q) = 1 - F(Q)$. Then it saves C_u.
- If demand is Q or less, the extra unit is left over. Chance: $F(Q)$. Then it costs C_o.
- So the extra unit is worth adding while $C_u \cdot (1 - F(Q)) > C_o \cdot F(Q)$.

The gain falls and the loss rises as Q grows, so you stop where they are equal:

$$C_u\,(1 - F(Q^*)) = C_o\,F(Q^*) \Longrightarrow F(Q^*) = C_u / (C_u + C_o) = \text{critical ratio}$$

In words: **stock up to the quantile of demand whose level equals the critical ratio.** That is why the forecaster gives quantiles: the answer is literally one of its lines.

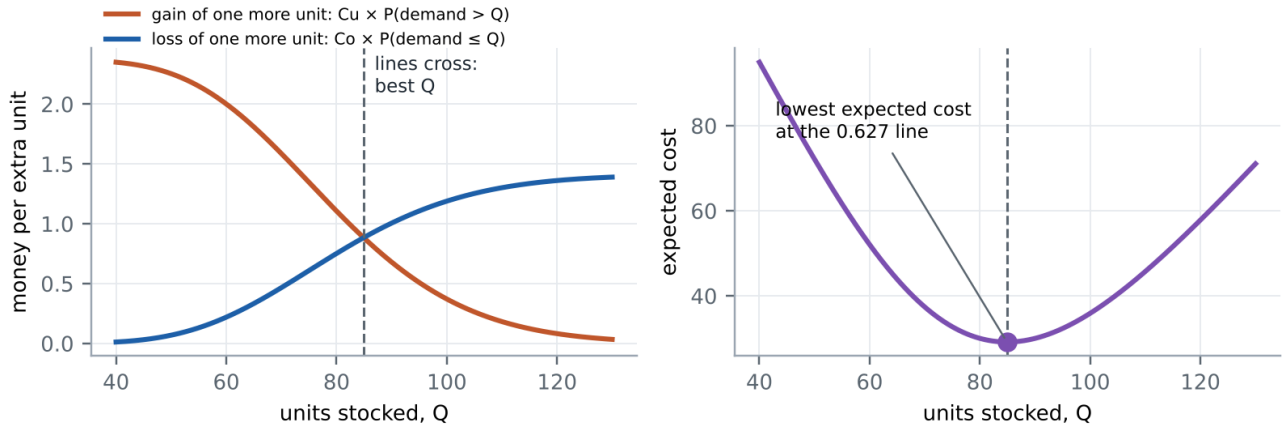


Illustration with a made-up 2-day demand of about 80 units and the costs of our example product. Left: the gain and loss of one more unit. Right: expected cost $E[C_u(D - Q)^+ + C_o(Q - D)^+]$, lowest where the lines cross.

For calculus fans: the expected cost is $E[C(Q)] = C_u \cdot E[(D - Q)^+] + C_o \cdot E[(Q - D)^+]$. Its derivative is $-C_u(1 - F(Q)) + C_o F(Q)$. Setting it to zero gives the same answer, and the second derivative $(C_u + C_o) \cdot f(Q) \geq 0$ shows it is a minimum.

What the ratio is for our products

Product type	C_u	C_o	Critical ratio	Line used
Perishable (any price p)	1.5p	0.85p × 1.05 = 0.8925p	1.5 / 2.3925 = **0.627**	between q50 and q80
Long-life	1.5p	only holding: 0.85p × 0.25/365 × 2 days ≈ 0.0012p	≈ 0.999	**q95** (highest line we have)

Because both costs are a fixed share of the price, the ratio is the same 0.627 for every perishable. For long-life items over-stocking costs almost nothing (the unit just waits a day or two), so the ideal line is far above q95; we use q95 because it is the highest line the forecaster makes.

Finding the 0.627 line between q50 and q80

We have no q63 line, so we draw a straight line between the two we have and read off 0.627 (linear interpolation):

$$\text{line} = q50 + (0.627 - 0.5) \,/\, (0.8 - 0.5) \times (q80 - q50) = q50 + 0.423 \times (q80 - q50)$$

The target for the cover window is the sum of the daily lines for today and tomorrow (closed days count as 0):

$$\text{target} = \text{line}_{\text{today}} + \text{line}_{\text{tomorrow}}$$

Two honest approximations. (1) The straight line between q50 and q80 is not the exact 0.627 quantile, but it is close because the curve is smooth there. (2) Adding two daily 0.627 lines gives slightly more than the 0.627 line of the 2-day total, because bad days rarely come together. So the target is a little on the safe side.

One-day view. The newsvendor model assumes leftovers are worthless, but a perishable with 4 days of life can still sell tomorrow. That is why waste is low and fill on perishables is the weak spot.

PART 5 · FROM TARGET TO NEED

Order-up-to, the spoilage cap and how waste is counted

The order-up-to rule

The **stock position** is everything the shop has or will get without a new order: stock on the shelf plus everything already ordered and on its way. The rule orders the gap between the target and that position:

$$\text{position} = \text{stock} + \text{on the way} \qquad \text{need} = \text{target} - \text{position}$$

If the need is zero or negative, nothing is ordered. In the store-only model yesterday's order has already arrived this morning, so 'on the way' is usually 0 and the position is simply this morning's stock.

The spoilage cap (perishables)

A product with shelf life L that arrives tomorrow must be sold within the next few days. So the shop should never hold more than the middle forecast of sales from today until today + L:

$$\text{cap} = \Sigma_{t\,=\,0\,\ldots\,L}\ q50_{\text{today}\,+\,t} \qquad \text{stock} + \text{order} \le \text{cap}$$

The forecaster only looks 2 days ahead, so days after tomorrow use the average of the two q50 values. The cap uses q50 on purpose: it limits stock to what we *expect* to sell, not what we might sell on a lucky day.

How the replay counts waste

The replay does not track the age of each unit. Instead, every evening it keeps only as many fresh units as the q50 forecast says will sell in the next L − 1 days, and counts the rest as thrown away:

$$\text{waste} = \max\left(\ \text{leftover} - \Sigma_{t\,=\,1\,\ldots\,L-1}\ q50_{\text{today}\,+\,t}\ ,\ 0\ \right)$$

This is the project's approved approximation. It is the same for every method, so comparisons stay fair.

The service floor

The newsvendor target can be cut by rounding or by a tight limit. The service floor promises that for every store-product the shop will hold at least 90% of its (capped) target:

$$\text{stock} + \text{order} + \text{gap} \ge 0.9 \times \min(\text{target}, \text{cap})$$

It is **soft**: if a limit makes it impossible, the gap variable takes up the slack, but every missing unit costs 10 × the lost-sale cost, so the model only breaks the promise when it really has no room.

From 'what is best' to a model a computer can solve

An optimisation model has three parts:

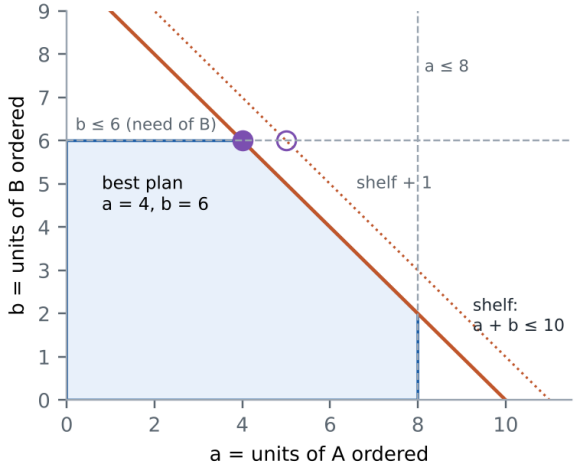
- **Decision variables**: the numbers we choose (how many cartons of each product).
- **Objective**: one number to make as small as possible (total cost).
- **Constraints**: rules every plan must obey (truck size, shelf space, budget).

If the objective and all constraints are straight-line (linear) formulas, it is a **linear program (LP)**. If some variables must be whole numbers, it is an **integer linear program (ILP)**.

A tiny LP you can draw

A shop needs 8 units of A and 6 of B. Each missing A costs 2, each missing B costs 3. There is room for only 10 units.

$$\text{minimise } 2(8 - a) + 3(6 - b) \text{ subject to } a + b \le 10,\ a \le 8,\ b \le 6,\ a, b \ge 0$$



The shaded area is every allowed plan (the **feasible region**). Missing a B costs more than missing an A, so fill B first (b = 6) and give the remaining 4 spaces to A. Cost = 2 × 4 = 8.

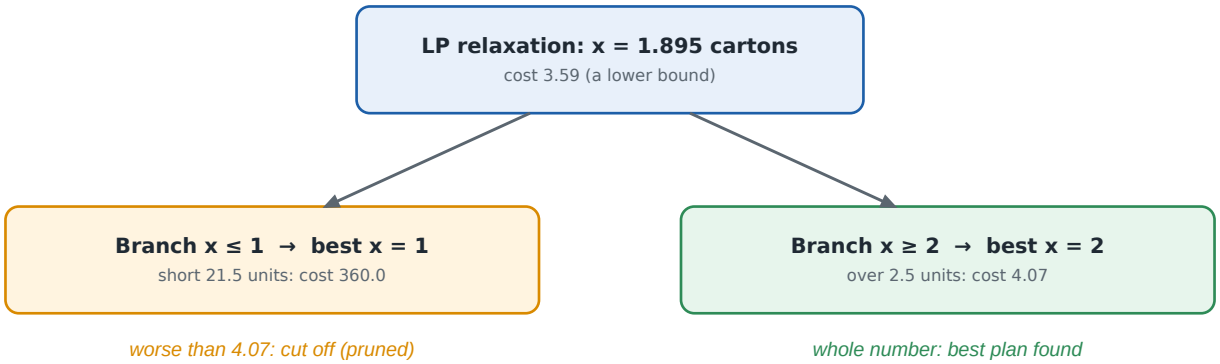
A basic LP fact: the best plan always sits on a **corner** of the region. The **simplex method** walks from corner to corner, always downhill, until no neighbouring corner is cheaper.

Keep the hollow circle in mind: with one more unit of space the best plan moves to a = 5 and the cost drops from 8 to 6. That drop of 2 is a **shadow price** (Part 8).

Why whole numbers make it harder

Cartons come whole. Solving the LP and rounding the answer can break a constraint or be far from the best whole plan. ILP solvers use **branch and bound**:

- Solve the LP ignoring the whole-number rule (the **relaxation**). Its cost is a lower bound: no whole plan can be cheaper.
- If a variable came out fractional, say x = 1.895, split the problem in two: one branch adds $x \le 1$, the other $x \ge 2$.
- Solve each branch. Any branch whose lower bound is already worse than the best whole plan found so far is thrown away (**pruned**). Repeat until every branch is solved or pruned.



The tree above is the real decision for our example product on 6 Nov (Part 9). The solver we use, CBC, does exactly this (plus extra tricks called cuts) for all 120 products of a store at once.

Two modelling tricks the store ILP uses

- **Splitting an absolute value.** We want to charge for being below the target and for being above it, at different prices. Write ordered − target = over − short with over, short ≥ 0. Because both carry a positive cost, at the optimum at most one is non-zero, so they measure exactly the shortfall and the excess.
- **A yes/no switch (big-M).** An order line costs 0.50 only if something is ordered. A binary z (0 or 1) with $x \le M \cdot z$ forces z = 1 whenever x > 0, and $x \ge 1 \cdot z$ enforces the one-carton minimum. M is the largest order that could make sense.

PART 7 · THE STORE ILP IN FULL

The one-day store ILP, line by line

One small ILP is solved each morning for each store. Index p runs over the store's products.

Decision variables

Variable	Type	Meaning
x_p	whole number ≥ 0	cartons of product p ordered today
z_p	0 or 1	1 if product p is ordered at all (pays the order-line cost)
y	whole number ≥ 0	trucks sent to the store
$short_p$, $over_p$	≥ 0	units below / above the newsvendor target
gap_p	≥ 0	units by which the 90% service floor is missed

Objective: minimise today's total cost

$$\min \Sigma_p \left[u_p \cdot short_p + o_p \cdot over_p + 0.5 \cdot z_p + 10 \cdot u_p \cdot gap_p \right] + 40 \cdot y$$

- u_p = 1.5 × price: cost of each unit below target (the newsvendor C_u).
- o_p = buy cost × 1.05 for perishables (C_o); for long-life items it is only the holding cost for the cover days (about 0.1% of the price per unit), so long-life items are almost always rounded up.
- 0.5 per order line, 40 per truck trip, and 10 × u per unit of broken service floor.

Constraints

Name	Formula	Plain meaning
Target	$upc_p \cdot x_p + short_p - over_p = need_p$	units ordered (cartons × units per carton) are compared with the need; short and over measure the miss
Order line	$1 \cdot z_p \le x_p \le M_p \cdot z_p$	at least one carton if ordering, and pay the line cost
Truck	$\Sigma x_p \le 400 \cdot y$	all cartons fit in the trucks sent
Shelf space	$\Sigma upc_p \cdot x_p \le$ free space tomorrow	free space = Σ shelf capacity − units still there tomorrow morning
Budget	$\Sigma cost_p \cdot upc_p \cdot x_p \le$ daily budget	1.25 × the store's average daily spend × open cover days
Service floor	$position_p + upc_p \cdot x_p + gap_p \ge 0.9 \cdot \min(target, cap)$	hold at least 90% of the target, or pay heavily
Spoilage cap	$position_p + upc_p \cdot x_p \le cap_p$	perishables only: never more than can sell before going bad

Shelf capacity of a product = max(days × its average daily sales, 2 cartons), with 10 days for long-life and 4 days for perishables (the teammate's original shelf rule). All numbers come from config.yaml and were fixed before any test run.

Size and speed

- Up to 6,010 variables per day, 2,410 of them whole numbers (all 10 stores together).
- The stores share no limit, so each store is solved as its own ILP. That gives the same answer as one big model, about ten times faster.
- Solved with CBC (through PuLP) to a 0.01% optimality gap; about 1 second per day on a laptop.

Why the target and the ILP are separate steps. The newsvendor rule answers 'how much is ideal for this product?'; the ILP answers 'given whole cartons and shared limits, what is the cheapest way to get close to all the ideals at once?'. Each step is simple to explain on its own, and the ILP's costs u and o are the same C_u and C_o, so the two steps agree on what 'cheap' means.

What is one more unit of shelf worth?

Every LP has a twin called its **dual**. The dual gives each constraint a price, the **shadow price**: how much the best cost would fall if that constraint's limit were loosened by one unit.

$$\text{shadow price of a limit} = -\,\Delta(\text{best cost}) \,/\, \Delta(\text{limit})$$

In the tiny LP of Part 6, raising the space from 10 to 11 lowered the best cost from 8 to 6, so space has a shadow price of 2 (one more A fitted in, saving 2). Raising the space from 20 to 21 would change nothing, because the shop already had room for everything: shadow price 0.

Complementary slackness

This is the key duality theorem for reading results: **if a limit is not full, its shadow price is 0; a limit can only have a positive price when it is full (binding)**. In symbols, for each constraint, slack × shadow price = 0.

How we get shadow prices from an ILP

Duals belong to LPs, not ILPs. So, as in the course roadmap, we: (1) solve the ILP; (2) fix the yes/no and truck variables (z and y) at their ILP values; (3) let the cartons x be fractional and re-solve as an LP; (4) read each constraint's dual from the solver. The result answers: 'with these trucks and order lines, what is one more unit of this limit worth today?'

Results at today's settings

Limit	Average use	Highest use	Shadow price
Truck (400 cartons)	21%	60%	0 on all 550 store-days
Shelf space	15%	32%	0 on all 550 store-days
Daily budget	35%	89%	0 on all 550 store-days

No limit is ever full, so by complementary slackness every shadow price is 0. That is a finding, not a bug: at the settings we assumed, the shops have plenty of truck, shelf and money.

The service-floor constraint does get prices (965 store-product-days). Its price is always 10 × u, the floor's penalty, because whenever the floor is short at all, each extra unit of floor costs exactly that penalty. Part 9 shows one. The spoilage cap binds only 6 times in 8 weeks, all on 23 Dec when the order must cover three days.

What-ifs: tightening one limit at a time

To see the limits matter, we re-ran the 8 weeks with one setting changed (core ILP, before the floor and cap layers):

Change	Fill	Waste	Total cost	Mean shadow price
Base	94.9%	8,043	934,088	0
Truck 60 cartons	94.8%	7,971	947,292	truck 0.0065 per carton
Shelf space × 0.3	86.1%	5,916	947,310	space 0.95 per unit per day
Budget × 0.45	93.0%	6,881	942,990	budget 0.50 per unit of money
Lost-sale penalty 1.0 × price	92.3%	5,498	not comparable*	0
Lost-sale penalty 2.0 × price	96.0%	10,112	not comparable*	0

* These rows price lost sales differently, so their totals cannot be compared with the base.

- With shelves cut to 30%, each extra unit of space would save about 0.95 a day, and fill falls to 86%.
- With the budget cut to 45%, each extra unit of money would save about 0.50.
- The setting that moves the plan most is the lost-sale penalty: from 1.0 to 2.0 × price, fill goes from 92.3% to 96.0% and waste from 5.5k to 10.1k. This is the newsvendor ratio at work: a higher C_u pushes the target line up.
- What-ifs are for understanding only; they were never used to choose settings.

Worked example: FOODS_3_252 at store CA_1

Everything below is real: real test-window sales, the forecasts the frozen forecaster already made for those days, and the orders the main model actually placed in its 8-week test replay. No holdout data is used.

The product

Fact	Value	Where it comes from
Product	FOODS_3_252, a perishable food item	items.csv (rank 3 by sales)
Shelf life L	4 days	items.csv
Carton size	24 units	case-size band for its sales rate
Price / buy cost	1.58 / 1.343 (85%)	median price, 28 days before the test
Average sales at CA_1	35.3 units a day	28-day average before the test
C_u = 1.5 × 1.58	2.37	lost-sale penalty
C_o = 1.343 × 1.05	1.410	buy cost + 5% disposal
Critical ratio	2.37 / (2.37 + 1.410) = **0.627**	Part 4

One morning in full: Friday 6 November 2015

Step 0, what the shop knows. Last night 15 units were left; this morning yesterday's 1 carton (24) arrived. Position = 15 + 24 = **39** units, nothing else on the way. The forecaster (using sales up to 4 Nov only) says:

Day	q50	q80	q95	0.627 line = q50 + 0.423 × (q80 − q50)
Fri 6 Nov (today)	29.92	39.43	45.63	29.92 + 0.423 × 9.51 = **33.94**
Sat 7 Nov (tomorrow)	45.55	57.31	63.49	45.55 + 0.423 × 11.76 = **50.53**

Step 1, target. 33.94 + 50.53 = **84.47** units for today and tomorrow (Saturdays sell more, and the forecast knows it).

Step 2, spoilage cap. Today to today + 4: 29.92 + 45.55 + 3 × 37.74 (the average of the two) = **188.66**. The target is far below it, so the cap does not bite.

Step 3, need. 84.47 − 39 = **45.47** units = 1.895 cartons. Service floor: the order must bring at least 0.9 × 84.47 − 39 = 37.02 units.

Step 4, the ILP's choice. The store's limits are far from full today, so this product's choice does not affect the others and we can check it by hand:

Cartons	Units	Below target	Above target	Floor missed	Cost
0	0	45.47 × 2.37 = 107.8	0	37.02 × 23.7 = 877.4	985.2
1	24	21.47 × 2.37 = 50.9	0	13.02 × 23.7 = 308.6	360.0 (incl. 0.5 line)
2	**48**	0	2.53 × 1.410 = 3.57	0	**4.07** (incl. 0.5 line)
3	72	0	26.53 × 1.410 = 37.41	0	37.91

The model orders **2 cartons**. This is the branch-and-bound tree of Part 6: the LP wanted 1.895 cartons; rounding down would leave 21 units short at 2.37 each, rounding up leaves 2.5 extra at 1.41 each, so up wins easily.

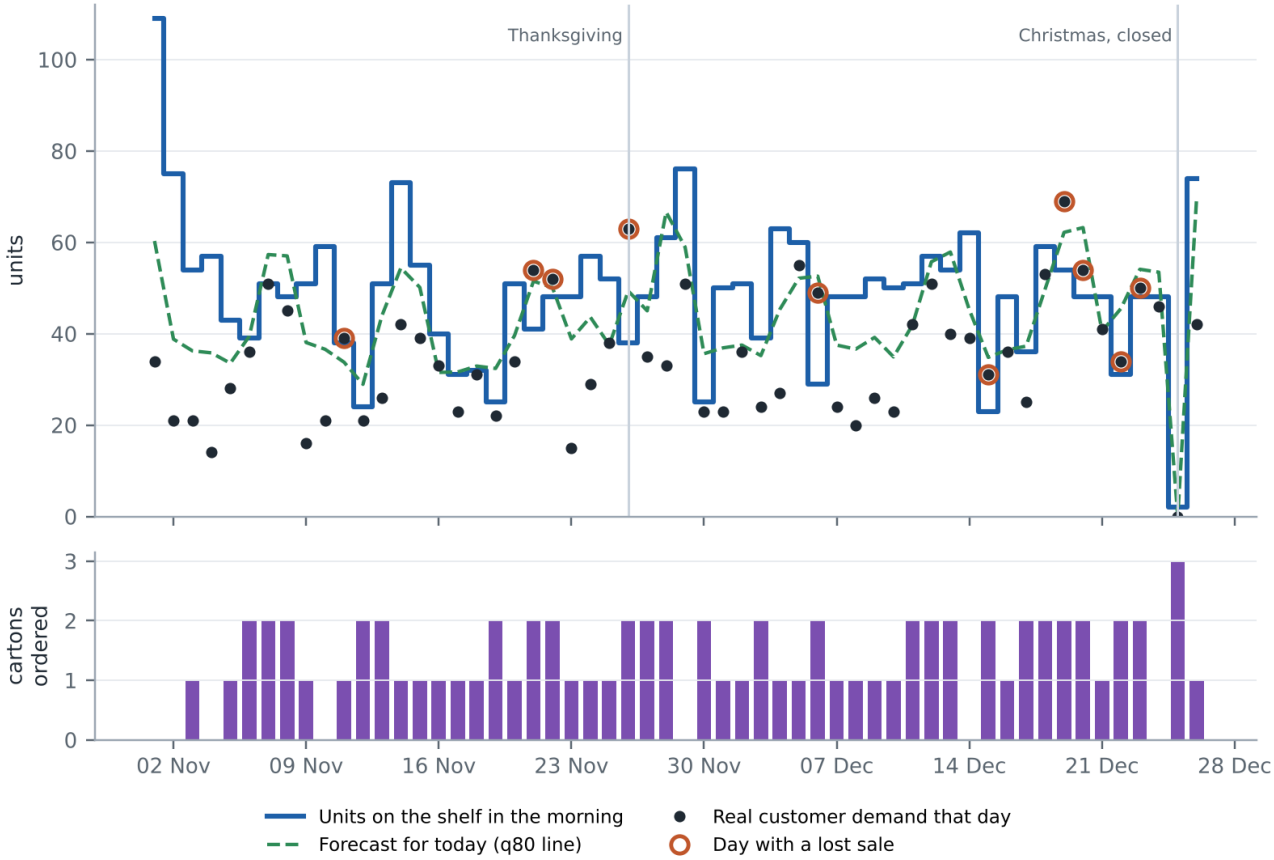
What really happened. Saturday morning the shop had 3 + 48 = 51 units. Customers wanted 51. All sold, nothing lost, nothing wasted.

> A hand calculation like this one, using only each day's forecast and morning stock, gives exactly the model's carton count on all 56 days for this product. That is a useful check, and it works because the store's shared limits never bind.

Four more days worth looking at

Day	What the model saw and did	Lesson
Wed 25 Nov → Thanksgiving (Thu 26)	Target 69.51, position 52, need 17.5 → 1 carton. Thursday's q50 was only 28.4 (q80 49.3). Real demand on Thanksgiving: **63**. Stock 38, so **25 units lost**.	The ILP can only be as good as the forecast. A holiday spike far above q80 still causes lost sales.
Sun 29 Nov	Position 76, target 84.45, need 8.45. 0 cartons: 8.45 short × 2.37 = 20.0. 1 carton: 15.55 over × 1.41 + 0.5 = 22.4. Ordered **0**.	A close call decided purely by C_u vs C_o. The 90% floor was missed by 0.007 units, so its shadow price that day was 10 × 2.37 = 23.7, the only floor price for this product.
23–24 Dec	23 Dec covers 23, 24 and 25 Dec (25th closed = 0): 2 cartons. 24 Dec: **no order allowed**, it would arrive on the closed day.	The calendar is built into the cover window.
Fri 25 Dec (closed)	Shop closed, but an order can be placed for the 26th. Target 60.0 (26 Dec only), position 2, need 58 → **3 cartons**. 26 Dec: 74 on shelf, demand 42, 32 left, no waste.	After the closure the model refills hard, because the newsvendor line says running out costs more.

The whole eight weeks



FOODS_3_252 at CA_1, 1 Nov to 26 Dec 2015. Top: morning stock (blue), today's q80 forecast (green dashes), real demand (dots); orange rings mark days with lost sales. Bottom: cartons ordered that morning (they arrive the next morning).

The stock line hugs the forecast: the model keeps roughly one to two days of sales on hand and re-orders 1 or 2 cartons almost every day. Lost sales happen on 10 of 55 open days, mostly busy weekends and Thanksgiving, when demand jumps above q80.

Same product, same 8 weeks, every method

Method	Fill	Lost units	Waste	Avg stock left at night	Cartons
Habit (order what sold last week)	90.2%	191	0	16.7	73
Top-up to forecast (best simple rule)	100%	0	13.3	46.8	82
Store model (rules only, no ILP)	92.6%	145	0	14.1	73
Main model (store ILP)	**95.0%**	**97**	**0**	**15.7**	**75**

For this product, top-up to forecast never runs out, but it holds three times as much stock every night and throws some away. The ILP beats the rule-only store model by deciding each carton by cost instead of a fixed round-to-nearest-carton rule: two extra cartons in 8 weeks saved 48 lost sales.

How it compares, and why one ILP beats patching

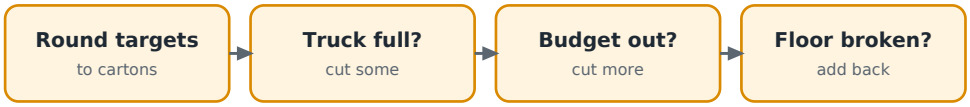
All methods, 8 test weeks

Method	Fill	Lost	Waste	Store stock value	Total cost
Habit: last week's sales	84.1%	81,426	19,908	18,062	1,015,406
Top-up to average + safety	95.2%	–	68.0k	28.3k	1,024k
Top-up to forecast	97.9%	10,608	73,861	33,845	1,012,368
Store model (rules only)	94.1%	30,282	6,842	35,240	937,832
Store ILP, core (no layers)	94.9%	26,208	8,043	35,279	934,088
Store ILP + floor + cap (MAIN)	**94.96%**	**25,727**	**7,015**	**35,663**	**932,094**

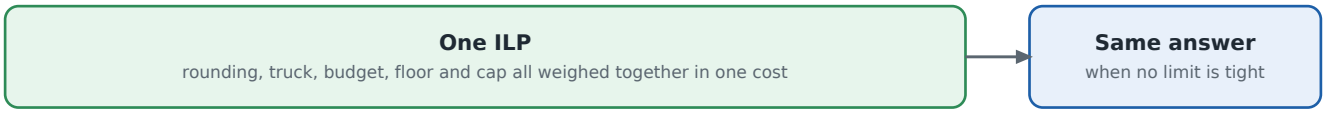
- The main model is the cheapest: 7.9% below the best simple rule and 0.6% below the rule-only store model.
- It wins on waste (90% less than top-up to forecast) and gives up some fill (97.9% → 95.0%). With a 1.5 × price penalty, the extra lost sales cost less than the waste and over-buying they avoid.
- Weak spots: perishable fill and average stock (similar to the rules).

Idea P1: patching rules one by one vs one ILP

(An explanation only. This was not built as a separate method.)



Patching: each rule fixes the plan in turn and never looks back at the earlier ones.



A simple way to handle limits is to patch the plan one rule at a time: round every target to cartons, then if the truck is too full cut some lines, then if the budget is exceeded cut some more, then add back anything that broke the 90% floor. Each patch fixes one problem but can undo the last one, and none of them asks which cut is cheapest overall.

The ILP weighs all the rules together in one cost. When something has to give, it cuts where the cost per unit is lowest (for example a slow long-life product with lots on the shelf, not a fast perishable).

When does it matter? Only when limits are tight. At today's settings no limit is ever full (shadow prices 0), so patching and the ILP would make almost the same plan. Under the what-ifs with shelf space at 0.3× or budget at 0.45×, limits bind on hundreds of store-days, and that is where solving everything together pays off. For the viva: the ILP is the right tool for a busy store; our test stores just happen to have spare room.

What the model does not capture

Limitation	What it means	Effect
Empty-shelf feedback is not modelled	The replay takes each day's recorded sales as the demand, whatever the shop shows. In real life an empty shelf changes behaviour: some customers switch product, come back tomorrow, or stop visiting. Also, the recorded sales were themselves limited by the real shop's shelves, so true demand on sold-out days was higher.	Lost sales are probably understated a little, and the cost of running out may be higher than 1.5 × price.
Waste is an approximation	Units are not tracked by age; waste is what exceeds the q50 sales of the next L − 1 days.	Same for every method, so the comparison stays fair.
Costs are assumptions	1.5 × price lost-sale penalty, buy cost 85% of price, 5% disposal, truck 40, line 0.50.	The penalty moves the plan most (Part 8 what-ifs).
One-day newsvendor	The model assumes leftover perishables are lost, but they may sell tomorrow.	Safe for waste, weak on perishable fill.
Sum of daily lines	Adding two daily 0.627 lines is slightly more than the 0.627 line of the 2-day total.	A little extra safety stock.
Store-only	The warehouse always has stock. Real warehouses can run short.	Fill is an upper bound; the warehouse switch is kept for later.
Limits never bind	Shelf, truck and budget sizes are assumptions built from averages.	Shadow prices are 0 at base; what-ifs show the effect.
Holdout pending	These are test-window numbers. The sealed holdout is being run now in 3 blocks.	Final claims wait for the holdout.

PART 12 · VIVA PRACTICE

Questions you may be asked

Q. Why not just order what the forecast says?

The forecast is uncertain and the two mistakes cost different amounts. The newsvendor rule picks the quantile where the cost of one more unit equals its expected saving: $F(Q^*) = C_u/(C_u+C_o) = 0.627$ for perishables, q95 for long-life.

Q. Where is the optimisation in this project?

The one-day store ILP: whole cartons and trucks (integers, branch and bound), truck/shelf/budget constraints, a soft service floor and a spoilage cap, minimising total cost. Then LP duality gives shadow prices, and what-ifs give sensitivity.

Q. Why are all your shadow prices zero?

Complementary slackness: a limit that is not full has price 0. Trucks are 21% full on average, shelves 15%, budget 35%. When we tighten them (shelf 0.3×, budget 0.45×) the prices become 0.95 and 0.50.

Q. How do you get duals from an integer program?

Fix the integer decisions (trucks y, order switches z) at the ILP optimum, relax cartons to continuous, re-solve the LP and read the duals. They are valid for small changes around that plan.

Q. Why is 0.627 the same for every perishable?

C_u = 1.5p and C_o = 0.85p × 1.05; the price cancels: 1.5 / (1.5 + 0.8925) = 0.627.

Q. Your fill is lower than the simple rule. Isn't that worse?

Fill is one side of the trade. Top-up to forecast throws away 73,861 units vs our 7,015 and costs 8% more in total. With the costs we set, the extra lost sales are cheaper than that waste.

Q. Can the model cheat by seeing the future?

No. Forecasts use sales at least 2 days old (rule R2), orders see only today's forecast, the replay applies the real day's sales afterwards, and the holdout is sealed.

Q. Why solve each store separately?

No constraint links two stores in the store-only design, so the big ILP splits into 10 independent ones with the same optimum, solved about ten times faster.

Q. What would you improve with more time?

Carry-over aware perishable targets (a multi-day newsvendor), modelling empty-shelf behaviour, real shelf and budget data so the limits mean something, and switching the warehouse back on.

Glossary

Term	Meaning
Quantile (q50, q80, q95)	A level sales stay at or below on that share of days
Critical ratio	$C_u/(C_u+C_o)$: the quantile the newsvendor model stocks to
Position	Stock on hand + orders on the way
Cover window	Days today's order must last: today and tomorrow
Spoilage cap	Most a perishable may hold: q50 sales over today … today + shelf life
Service floor	Hold at least 90% of the target, or pay 10 × the lost-sale cost per unit
LP / ILP	Linear program / the same with some whole-number variables
Relaxation	The ILP with the whole-number rule dropped; gives a lower bound
Branch and bound	Split on a fractional variable, solve both halves, prune hopeless ones

Term	Meaning
Dual / shadow price	Value of one more unit of a limit; 0 when the limit is not full
Complementary slackness	Slack × shadow price = 0 for every constraint
What-if	Re-run with one setting changed, to see how sensitive the plan is
Fill rate	Units sold ÷ units demanded